Task 6: Robustness, Pull Request, Demo, and Handoff

Phase 2, Task 6 deliverable. Author: Ahmed. Date: 2026-11-13. Repository: https://github.com/pathlyworks/brandstore-ecommerce · Pull request: **#3** (`feature/robustness-handoff → main`) · Description: docs/PR_TASK6.md.

Part	Where
Edge-case sweep (5 cases, each with automated tests)	§1 below; tests in `tests/unit/edge-sweep.test.js`, `tests/dom/edge-sweep-ui.test.js`, `tests/e2e/specs/robustness.spec.js`
Keyboard and accessibility pass (catalog, cart, live regions)	§2; `tests/e2e/specs/keyboard-checkout.spec.js`, the accessibility block of `tests/dom/edge-sweep-ui.test.js`
Pull request and one merge conflict resolved against `main`	§3; PR #3, merge commit `01fb719`
Cross-browser smoke test	§4; CI jobs "Cross-browser smoke (firefox / webkit)"
Lighthouse	§5; `docs/lighthouse/README.md`, CI job "Lighthouse"
README (setup, architecture, decisions, tests)	README.md
Demo video (3–5 minutes): happy path and three failure paths	docs/demo/brandstore-demo.mp4, chapters in `docs/demo/chapters.md`
Handoff notes: API contract gaps for the backend team	docs/HANDOFF.md (16 gaps, 4 open questions)

1. Edge-Case Sweep

Every case is handled in the app and proven by automated tests at three levels: pure logic (Vitest), the real page in jsdom with Mock Service Worker, and a real browser against the TechSphere API (Playwright, in Chromium, Firefox and WebKit in CI). Test names begin with "edge N", so each row can be found by searching for it.

#	Case	How the app handles it	Evidence (test names)
1	**Missing product image**	A product with no images, an empty list, `null` or non-text entries gets the placeholder, with the product name as alt text. Root-relative paths to the API's `/images` endpoint are not loaded (the spec says that endpoint answers JSON). An absolute image URL that fails in the browser (404, not an image) is replaced once by the placeholder, with no retry loop (`utils/dom.js` `withImageFallback`).	Unit: *edge 1: missing product image* (3 tests). jsdom: *edge 1: … shows the placeholder … when the image fails*; *… /images endpoint … is not loaded at all*. E2E: *edge 1: a missing product image shows the placeholder, never a broken image* (kit's p-010, then a real 404 URL; checks `naturalWidth > 0`). Demo at 3:09.
2	**Null specification fields**	Rows whose value is `null`, `undefined` or empty are left out. Keys become labels ("battery_life" becomes "Battery life"). With no rows left, the page says "No specifications are listed for this product." It never shows "null" or an empty table, for either API format (`api/techsphere.js` `toAppSpecs`, `components/product-detail.js` `createSpecs`).	Unit: *edge 2: null specification fields* (3 tests). jsdom: *edge 2: specs that are all null show one clear line …*; *… a product with real specs still shows its table*. E2E: *edge 2: null specification fields are left out, never shown as "null"* (the API answer is rewritten to `{ warranty: null, weight: null }`).
3	**429 rate limit**	Catalog, product page and cart each explain the 429 and offer **Try again**. The wait comes from `Retry-After` when the API sends it, otherwise "a moment". The cart keeps its stored prices and says they could not be checked. Placing an order on a 429 keeps the cart: "Your order was not placed. Too many requests …".	Unit: *edge 3: 429 rate limit* (3 tests). jsdom: *catalog … 429* and *cart … 429* (Task 5); *edge 3: the product page explains the 429 …*; *… a 429 while placing an order keeps the cart*. E2E: *edge 3: a 429 on the catalog explains itself, and "Try again" recovers*. Demo failure path 2 (2:33).
4	**Empty-cart checkout attempt**	"Place order" is `aria-disabled` on an empty cart: still focusable, so keyboard and screen-reader users can reach it and hear why. Pressing it says "Your cart is empty. Add a product before you check out." and **sends no request**. The cart is checked again on every press, so a cart emptied in another tab is caught too. A cart with only out-of-stock lines is stopped the same way (`state/cart.js` `checkoutItems`).	Unit: *edge 4: empty-cart checkout attempt* (3 tests). jsdom: *edge 4: "Place order" on an empty cart explains itself and sends nothing*; *… a cart emptied in another tab …*; *… with items: signed out goes to sign-in …*. E2E: *edge 4: "Place order" on an empty cart explains itself and sends no request* (records every request). Demo failure path 3 (2:55).
5	**Offset-versus-page pagination mismatch**	`GET /products` pages by `page` (always 8); `GET /account/orders` by `offset` and `limit`. Both become the app's `{ page, limit, total, totalPages }`. The page size is fixed at 8, so a short last page counts from the right number. A page past the end (`?page=9` of 3) moves to the last page. App pages become offsets (`page 2 → offset=5`), and an offset that is not a multiple of the limit lands on the page that holds it.	Unit: *edge 5: offset-versus-page pagination mismatch* (4 tests). jsdom: *edge 5: a page past the end (?page=9) moves to the last page …*. E2E: *edge 5: a page past the end (?page=9, the API pages by 8) moves to the last page*; *sign in … and view the orders* (Next page = offset 5). Demo at 3:17 and 1:59.

Bugs the sweep found and fixed:

- Specs with `null` printed "null".
- On a short last page the count started at the wrong number: `limit` was taken from `items.length`.

- An empty cart showed "$0.00" against the EGP-priced API.
- The price filter legend said "USD".
- No checkout existed, so there was nothing to stop an empty-cart attempt (added: POST /orders).

2. Keyboard and Accessibility Pass (Catalog and Cart)

Check	Result	Evidence
Tab order on the catalog	Skip link first, then the menu, search, one stop per radio group (arrow keys inside), price, rating, sort, the products, pagination	E2E *keyboard: the skip link, filters, products and cart controls are all in the tab order*
Whole journey by keyboard only	Filter (Space), open a product (Enter), add to the cart, change the quantity, sign in (typing, Tab, Enter), place the order	E2E *keyboard only: filter, add to cart, change quantity (announced), sign in and place the order*
Cart updates announced (`aria-live`)	Every change says what happened and the new total in the cart's `role="status"` region, e.g. "Quantity of Orbit Audio 3 is now 2. Cart total: EGP 4,998.00 for 2 items." Remove, save for later and clear cart are announced too, as are price changes found on load	jsdom *accessibility: cart updates are announced in the live region*; the E2E above checks the text in a real browser
Focus after a cart re-render	Focus returns to the control that was used (the + button), or to the cart heading if that line is gone	E2E above (`toBeFocused`); Task 3 journey
"Add to cart" result	Announced in the product page's status region with a "View cart" link	E2E *filter the catalog … add a product to the cart*
Checkout button	`aria-disabled` rather than `disabled` (focusable) and `aria-describedby` its note	E2E *edge 4* (`aria-disabled="true"`, keyboard Enter)
Errors	Catalog and product errors use `role="alert"` with a heading and a Try again button. Form errors use `aria-invalid`, `aria-describedby`, and a summary that receives focus	jsdom catalog/product tests; Task 4 journeys
Images	Every product image has alt text (the product name); the cart's thumbnails are decorative (`alt=""`)	jsdom *edge 1*
Contrast and keyboard (Phase 1 checks)	Lowest contrast ratio 5.02; menu, carousel and Escape behaviour pass	`node qa/contrast.js`, `node qa/keyboard.js`: 0 failures
Lighthouse accessibility	**100** on home, catalog, product and cart, mobile and desktop	§5

Fixed in this pass:

- Quantity, remove and save-for-later changes were silent for screen readers.
- The disabled Checkout button could not be reached with the keyboard.

3. Pull Request and Merge Conflict

Workflow on GitHub (repository `pathlyworks/brandstore-ecommerce`, public):

PR	Branch → main	What	Merged
#1	`feature/automated-tests-ci`	Task 5: tests and CI. The first real CI run caught one bug (Vite rewrote `new URL(…, import.meta.url)` in the jsdom tests); it was fixed in the PR	On GitHub, "Create a merge commit"
#2	`fix/cart-shipping-copy`	A teammate's copy change in `cart.html`: currency-neutral shipping rule, a clearer checkout note, a returns line	On GitHub, "Create a merge commit"
#3	`feature/robustness-handoff`	**This task**	On GitHub, "Create a merge commit", after green checks

The conflict. PR #2 landed on `main` while this branch rewrote the same lines of `cart.html` (the order summary). `git merge origin/main` stopped with:

```
<<<<<<< HEAD (this branch)
<p class="totals__note">
  Shipping is free on orders of $100.00 or more, otherwise $7.99. No tax is charged.
</p>
<button
  type="button"
  id="checkout"
```

```
  class="btn btn--secondary"
  aria-describedby="checkout-note"
  aria-disabled="true"
>
  Place order
</button>
<p id="checkout-note" class="totals__note">Add a product to your cart to check out.</p>
======= (main, PR #2)
<p class="totals__note">
  Shipping is free on orders of 100.00 or more in your cart's currency, otherwise 7.99. No tax is
  charged.
</p>
<button type="button" class="btn btn--secondary" disabled aria-describedby="checkout-note">
  Checkout
</button>
<p id="checkout-note" class="totals__note">
  Checkout opens soon. Your cart is saved on this device, so nothing is lost meanwhile.
</p>
<p class="totals__note">Free returns within 14 days of delivery.</p>
>>>>>>> origin/main
```

Resolution (merge commit `01fb719`), keeping both intents:

Line	Kept from	Why
Shipping rule	`main`	Product's currency-neutral wording; this branch had not changed it on purpose
Button	this branch	The working "Place order" (id, `aria-disabled`, `aria-describedby`) replaces the disabled placeholder that `main` still had
Checkout note	both	"Checkout opens soon" stops being true once checkout exists. The promise behind it ("Your cart is saved on this device, so nothing is lost meanwhile") is kept in the note's text and in the signed-out note in `components/checkout.js`
Returns line	`main`	New content; kept as is

The test that reads the note was updated in the same commit. The merged branch passed lint, all test levels and all CI jobs (§6).

4. Cross-Browser Smoke Test

The Playwright specs (catalog and cart, sign-in and orders, the five edge cases, the keyboard journey) run in CI on every pull request in Chromium, Firefox and WebKit (`E2E_BROWSERS`, `playwright.config.mjs`):

Browser	Engine	Result on PR #3	Notes
Chrome / Edge	Chromium	**9 passed**	Also the 71-run journey suite (desktop and phone)
Firefox	Gecko	**9 passed**	The first five runs failed with "Page crashed" in the catalog test. Probes in CI isolated it to Playwright's `allTextContents()` on a `getByRole` locator in its patched Firefox. Mouse and keyboard filtering, `check()` and `evaluateAll()` all pass. The test now uses a web-first `toHaveText()`. Two app changes made while searching (not loading the API's JSON "images"; updating the radio filters in place) stay, because each holds up on its own
Safari	WebKit	**8 passed, 1 skipped**	The tab-order check is skipped: WebKit only tabs to links when the macOS "Press Tab to highlight each item" setting is on. The keyboard-only journey still runs

5. Lighthouse

Lighthouse 12.6.1 in CI on the **built** site (`npm run build`) served against the TechSphere API. Scores are 0–100:

Page	Form factor	Performance	Accessibility	Best practices	SEO
home	mobile	98	100	100	100
catalog	mobile	91	100	100	100
product	mobile	94	100	100	100
cart	mobile	95	100	100	100
home	desktop	100	100	100	100
catalog	desktop	100	100	100	100
product	desktop	100	100	100	100
cart	desktop	100	100	100	100

Accessibility, Best practices and SEO are 100 everywhere. The mobile performance points left are layout shift on the catalog (0.143) and the cart (0.112), compression and caching (host settings: Netlify compresses; long cache lifetimes would need fingerprinted file names), and render-blocking CSS (no bundler by design). Details: `docs/lighthouse/README.md`.

Full HTML and JSON reports are in the CI artifact "lighthouse" of the run (30 days); the summary is in `docs/lighthouse/README.md`. Before this task the catalog loaded 8 image URLs per page from the API; removing them is the main performance change.

6. Checks on the Final Commit

CI run 43459653927, on the last code change of PR #3. Every job passed; later commits add only documents and the demo video, and their runs are green too:

Job	Result
Lint, format and build	ESLint and Prettier clean, branch-name check, `npm run build`, contract: 58 requests, 10 operations, 0 problems
Unit and integration tests (Vitest, jsdom, MSW)	**259 of 259** tests in 26 files; coverage of lines **src/state 99.86%** (714/715), **src/utils 99.00%** (794/802)
End-to-end tests (Playwright, Chromium)	**9 of 9** specs; journey suite **71 of 71** runs; 197 API requests checked against the spec, 0 problems
Cross-browser smoke (Firefox)	**9 of 9**
Cross-browser smoke (WebKit)	**8 passed, 1 skipped** (tab order; see §4)
Lighthouse	§5

Also run locally: `qa/js-rules`, `preflight` (475 passed), `headers`, `behaviour`, `keyboard`, `layout` and `contrast` all have 0 failures, and CLS is 0 at desktop size.

7. Demo Video

`docs/demo/brandstore-demo.mp4`, 3 minutes 38 seconds, 1280×720, with captions. It was recorded by `scripts/record-demo.mjs` in Chromium against the TechSphere API kit's server, and can be re-recorded with `node scripts/record-demo.mjs`. Chapters: `docs/demo/chapters.md`.

Part	Shows
Happy path (6 steps)	Catalog, search, category filter, sort, product page, add 2 to the cart, quantity change announced, Place order → sign in → back to the cart → order placed (POST /orders 201), account orders with Next page, sign out (cart kept)
Failure path 1	Wrong password: one clear message, focus moved to it, no password in the URL
Failure path 2	The API answers 429 (forced through the kit's control endpoint): "Too many requests", Try again, recovered
Failure path 3	Place order with an empty cart: explained, no request. Then the missing image and the past-the-end page

8. Handoff to the Backend Team

`docs/HANDOFF.md` lists 16 API contract gaps. Each gives what we observed, the front end's workaround and the request, ordered by risk. The biggest:

1. Prices come as numbers or as formatted strings.
2. Stock is a status with no quantity, and sometimes missing.

3. `POST /orders` validates nothing and returns no totals.
4. Two pagination schemes.
5. Four date formats.
6. Images that are JSON.
7. Errors with no codes and no `Retry-After`.

It ends with four questions the front end needs answered.

Appendix: Handoff notes — API contract gaps (docs/HANDOFF.md)

From: BrandStore front end (Ahmed). **To:** TechSphere API team. **Date:** 2026-11-13. **API:** TechSphere Store API 1.1 (`api/techsphere/openapi.yaml`, mock `server.py`).

Below is every place where the API's real responses differ from its spec, or where the spec leaves the front end guessing. Each item gives what we observed, how the front end works around it today (the code lives in `src/api/techsphere.js` unless another file is named), and what we are asking for. Items are ordered by risk: the first ones can show a shopper a wrong price or let them order something unavailable.

#	Area	Observed	Front-end workaround today	Request
1	**Price type**	`price` is a number in pounds (`32999`) for most products, but a formatted string for some (`"EGP 33,999"` on p-005 and p-014). Order `total` is a number in pounds.	`priceToCents` strips everything but digits and the dot, then works in integer cents. An unreadable price becomes `NaN`, never 0.	One type: an integer in minor units (`3299900`) plus `currency`, on products and orders.
2	**Stock**	Only `stock_status` (`in_stock`, `low`, `out_of_stock`), and it is missing on some products (p-007, p-018). There is no quantity.	Missing is treated as in stock. Assumed quantities are `in_stock` 25, `low` 3, `out_of_stock` 0, and the cart limits use them.	Make `stock_status` required and add `stock_quantity` (or `max_per_order`).
3	**Placing an order**	`POST /orders` accepts any `productId` and `qty`: an unknown product, an out-of-stock product and a huge quantity all get 201. The answer has no prices or total.	The app sends only in-stock lines and never sends an empty cart (`checkoutItems` in `state/cart.js`). It cannot know the price that was charged.	Validate products and stock (409 with the failing lines). Return the order with lines, prices and total. Accept an `Idempotency-Key` header so a double click cannot create two orders (see 11).
4	**Pagination**	Two schemes. `GET /products` pages by `page`, always 8 a page, and ignores `limit`. `GET /account/orders` pages by `offset` and `limit` (default 5). A page past the end returns `items: []` with the real `total`.	Both are mapped to `{ page, limit, total, totalPages }`. The page size is fixed at 8 (`TECHSPHERE_PAGE_SIZE`). Past the end, the catalog moves to the last page.	One scheme for every list (cursor or `page` plus `page_size`), with `page_size` in the response.
5	**Dates**	Order `created_at` comes in four formats: `2026-08-01T10:15:00Z`, `02/08/2026`, `2026-08-03T18:40:00+03:00` and `4 Aug 2026`. `02/08/2026` is ambiguous (2 August or 8 February).	`utils/dates.js` reads all four. `dd/mm/yyyy` is assumed.	ISO 8601 in UTC only.
6	**Images**	`images` are root-relative paths (`/images/p-001.webp`) with no stated base. The spec says `GET /images/...` answers a JSON placeholder (`{"image": ...}`), not image bytes. Missing images (p-010, p-021) answer 404. There is no alt text.	Root-relative paths are not loaded: they would cost 8 requests a page towards the rate limit and show nothing. Our placeholder is shown, with the product name as alt text. Absolute URLs are loaded, and any image that fails is replaced once by the placeholder (`utils/dom.js`).	Absolute image URLs (or a documented CDN base) that serve real images. Return `null` instead of a path that 404s. Add alt text.
7	**Specifications**	`specs` is a free-form object (`{ "warranty": "1 year" }`) with lower-case keys and no units. The spec allows any value, so `null` is possible.	Rows with `null`, `undefined` or empty values are left out. Keys become labels (`battery_life` becomes "Battery life"). With no rows left, the page says "No specifications are listed".	`[{ label, value, unit }]` with no empty values, or document that `null` means "unknown".
8	**Errors**	Bodies are `{ "error": "text" }` only: no machine code and no per-field details. The same 400 covers an invalid email, a short password and "Email already registered".	Codes are guessed from the status (`toAppError`). "Already registered" is found by matching the English text.	`{ "error": { "code", "message", "details": [{ "field", "message" }] } }`, and 409 for "already registered".
9	**Rate limiting**	429 is not in the spec at all. When it happens, there is no `Retry-After` header.	Every page handles 429 with "Too many requests … Wait a moment, then try again" and a Try again button. With `Retry-After`, the app would count it down.	Document 429 on every endpoint and send `Retry-After`.
10	**Sign-in**	`POST /auth/login` answers 401 for everything, including a missing email or a body that is not JSON.	The form validates first, so the user sees field errors before anything is sent.	400 with field details for a malformed request; keep 401 for wrong credentials.
11	**CORS**	`Access-Control-Allow-Headers` is only `Authorization, Content-Type`.	The app does not send its `X-Client-Id` header to TechSphere (it would be blocked).	Also allow `Idempotency-Key` (and `X-Request-Id` for support), or list the headers that may be used.
12	**Tokens**	The JWT is unsigned (`alg: none`, signature `mock`). It lasts 1 hour, with no refresh token and no sign-out endpoint.	The token is kept in sessionStorage. `exp` is read so the account page can leave when the token expires. An expired session or any 401 goes back to sign-in with a message.	A signed token. A refresh flow, or an HttpOnly cookie (then CSRF protection is needed; see `docs/SECURITY.md`). `POST /auth/logout` to revoke.

#	Area	Observed	Front-end workaround today	Request
13	**Addresses**	There is no `POST` to create an address. `PUT` merges any fields sent into the stored address, not only `city` and `street`. `DELETE` answers 204 for an id that does not exist.	The address book only edits and deletes. The app sends only `city` and `street`.	Add `POST /account/addresses`. Reject unknown fields with 400. Answer 404 for an unknown id.
14	**Order lines**	Orders list `items: [{ productId, qty }]` with no name or price.	Order history shows product ids and quantities. Fetching every product would cost one request per line and risk a 429.	Include the name and unit price in each order line (a snapshot at order time).
15	**Re-checking the cart**	There is no batch lookup, so re-checking prices and stock takes one `GET /products/{id}` per cart line.	Requests run in parallel. Failures keep the stored line and say so.	`GET /products?ids=p-001,p-002`.
16	**Names**	Search is `search` here but `q` in most APIs we use. The sort value is `rating`, not `rating_desc`. There is no name sort.	Translated in `toTechsphereRequest`. "Name: A to Z" is not sent to TechSphere.	Fine to keep, but please add `name_asc` and freeze the names in the spec.

What the front end already guarantees

- The address comes only from `VITE_API_BASE`, falling back to `http://localhost:4010`. No other URL is fixed in the code (`src/config.js`).
- The token travels only in the `Authorization` header: never in a URL, a query string or a log (`docs/SECURITY.md` §6).
- Every request and response mapping above is covered by unit tests (`tests/unit/techsphere.test.js`, `tests/unit/edge-sweep.test.js`). End-to-end tests run against `server.py` on every pull request.

Questions we need answered

1. Is `02/08/2026` 2 August or 8 February? (Item 5.)
2. Should a product without `stock_status` be sold? (Item 2.)
3. What should `POST /orders` do when a line is out of stock: reject the whole order, or place the rest? (Item 3.)
4. Will production send `Retry-After`, and how many requests per minute are allowed? (Item 9.)